Code Review: `rag/` SOP Retrieval Platform

Cold-Chain Logistics AI-Assistant · Review date: 13 Nov 2026 · Status: Draft for team review

Item	Detail
Branch	`feat/langgraph-secure-agent` (base commit `c2caa93`)
Code reviewed	`rag/` (17 files, 1,299 lines), uncommitted working tree, content SHA-256 `20a79f80b25e`
Reproduction tests	`tests/review_findings_test.py`

1. Verdict and Summary

> **Verdict: NO-GO.** Do not deploy `rag/` until **F-01** and **F-02** are fixed. Both fixes are small (estimated under a day combined), and the rest of the package is in good shape.

The review raised **10 findings**. Each was turned into an automated reproduction test: **9 were confirmed** and **1 was withdrawn** (F-08), because the Pinecone SDK source showed the suspected race cannot happen.

Severity	Count	Findings
Blocker	2	F-01, F-02
High	1	F-07
Medium	4	F-03, F-04, F-06, F-09
Low	2	F-05, F-10

Why no-go. The two blockers undermine the two guarantees an operator relies on most:

- **F-01:** an SOP that parses to no text is silently removed from search, and the run still reports success.
- **F-02:** a broken configuration, such as a bad API key, makes the scheduled ingestion job exit as if it were a routine partial failure, so nobody is alerted.

Either one can leave the agent answering "no matching SOP" for procedures that exist, with nothing in the logs to explain why.

What is good. The architecture is sound and largely matches the Technical Design Document (TDD): a read/write split, a read-only retrieval path, upsert-then-prune updates, atomic manifest writes, retries only on transient errors, and an offline test suite (Section 7). Most findings are edge cases that one of those designed guarantees misses, not structural flaws.

Path to GO. Fix F-01 and F-02, move their reproduction tests into the main suite, and keep it green (Section 8). Fix F-07 before the LangGraph agent is wired to the retrieval tool.

2. Scope and Method

In scope. The `rag/` package, all 17 Python files:

- `rag/config.py` : typed settings and the pipeline fingerprint.
- `rag/core/` : the embedder wrapper, the Pinecone gateway, the retry policy and the data models.
- `rag/ingestion/` : the offline write path (CLI, plan, pipeline, manifest, parsers).
- `rag/retrieval/` : the online read path (service logic, FastAPI app, and the agent-side client and LangChain tool).

Out of scope. These were not reviewed:

- The legacy MSSQL loader (`scripts/` , `legacy_ingest/`).
- The LangGraph agent and Streamlit UI, which don't exist yet.
- Infrastructure and deployment configuration.
- Dependency management.
- A dedicated security review.

The review covered **correctness** only, not style.

Method.

1. **Static review.** Every file in scope was read end to end, looking for paths where the code's behaviour differs from its own documented contract (docstrings, exit codes, the design document's resiliency rules) or from how its dependencies actually behave.
2. **Reproduction.** Each candidate finding was turned into a unit test that asserts the *correct* behaviour. The test runs against the real `rag/` code with test doubles at the boundaries: - an in-memory Pinecone index; - a deterministic hash-based embedder; - `httpx.MockTransport` ; - FastAPI's `TestClient` .

A finding counts as **confirmed** only if its test fails for the reason described. 3. **Falsification.** Where a finding depended on how a third-party library behaves, the installed library source was checked. This is how F-08 was withdrawn (Section 6).

Limitations.

- All tests run offline. Pinecone, OpenAI and the bge-m3 model are simulated, so Pinecone error objects are modelled as exceptions with a `status` attribute, matching the SDK's convention.
- Performance (F-10) is measured by counting operations, not by timing benchmarks.
- `rag/` is not yet committed. The content hash in the header identifies the exact code that was reviewed.

3. Severity Rubric

Severity	Definition	Deployment rule
Blocker	Causes silent data loss, or hides a failure from operators or alerting, under the **default** configuration.	Must be fixed before any deployment.
High	Wrong or crashing behaviour on a path the agent or operators depend on.	Must be fixed before the agent integration ships.
Medium	Wrong behaviour that needs a non-default setting, uncommon input, a direct API caller, or a deployment mistake to trigger.	Fix in the hardening pass.
Low	Input validation gaps or performance issues that don't matter at current scale.	Fix when convenient.

Effort: S means under half a day, including the test. **M** means about one day.

4. Findings at a Glance

ID	Finding	Component	Severity	Effort	Status	Reproduction test
F-01	A document with no text silently wipes its vectors	ingestion	Blocker	S	Open	`Finding01_EmptyDocumentWipesIndex`
F-02	Config errors crash instead of exiting with code 2	ingestion CLI	Blocker	S	Open	`Finding02_ConfigErrorsExitWith2`
F-07	Agent tool misreports caller errors and crashes on malformed responses	retrieval client	High	S	Open	`Finding07_ClientMisreportsErrors`
F-03	`EMBEDDING_DIMENSIONS` can provision an unusable index	core / embeddings	Medium	S	Open	`Finding03_EmbeddingDimensionsOverride`
F-04	Spreadsheet rows ignore `chunk_size`	ingestion / parsers	Medium	S	Open	`Finding04_TableRowsNotSplit`
F-06	Invalid filter reported as a 503 outage	retrieval API	Medium	M	Open	`Finding06_BadFilterReportedAsOutage`
F-09	`/readyz` passes with an empty namespace	retrieval API	Medium	S	Open	`Finding09_ReadyWithEmptyIndex`
F-05	`--workers` bypasses validation	ingestion CLI	Low	S	Open	`Finding05_WorkersNotValidated`
F-10	Plan building is O(n^2)	ingestion	Low	S	Open	`Finding10_QuadraticPlan`
F-08	Index recreate races Pinecone deletion	core / vector store	n/a	n/a	~~Withdrawn~~	`Finding08_RecreateRace_Withdrawn` (passes)

5. Detailed Findings

Each finding has a reproduction test in `tests/review_findings_test.py`. The test asserts the *correct* behaviour, so it fails while the finding is open and passes once the finding is fixed. The evidence for each finding is the actual output of that test against the reviewed code.

F-01 · A document with no text silently wipes its vectors

Severity: Blocker · **Component:** ingestion · **Status:** Open, reproduced · **Effort:** S

Location: `rag/ingestion/pipeline.py:153–154`, `:180–184`

Problem. When a document parses to zero chunks, the pipeline only logs a warning and carries on as if it had succeeded:

```
chunks = parse_document(path, chunk_size=..., chunk_overlap=...)
if not chunks:
    logger.warning("%s: no extractable text (scanned PDF? needs OCR). Indexing 0 chunks.",
name)
```

Nothing is upserted, so the stale-chunk pruning step (line 180) treats *every* existing vector of the document as stale and deletes it. The manifest then records the file as current with `chunk_count=0`. This contradicts the rule stated in `parsers.py`: a broken file must be "reported and retried rather than silently recorded as 'ingested with zero chunks'".

Impact. A scanned PDF, an image-only SOP, or an SOP saved blank by mistake removes that procedure from search, and the run still exits 0. The manifest treats the file as up to date, so it is never retried until its contents change. Without an error in the logs, the agent starts answering `NO_MATCHING_SOP` for that procedure. This is exactly the "previous version stays searchable" guarantee the pipeline is designed to provide.

Evidence. `Finding01_EmptyDocumentWipesIndex`. After ingesting a 2-chunk SOP and overwriting it with whitespace: vectors went from 2 to 0, `report.ok == True`, `report.failed == {}`, and the manifest recorded `chunk_count == 0`.

Recommended fix. In `_ingest_document`, raise `DocumentParseError` when `chunks` is empty. The existing per-document error handling then marks the file failed, keeps its previous vectors and retries it on the next run. If an intentionally empty file must be supported, make that an explicit opt-in.

F-02 · Configuration errors crash instead of exiting with code 2

Severity: Blocker · **Component:** ingestion CLI · **Status:** Open, reproduced · **Effort:** S

Location: `rag/ingestion/cli.py:32`, `:74–78`

Problem. The CLI documents exit code 2 for configuration errors, but it only catches two exceptions:

```
settings = get_settings()          # line 32: ValidationError escapes
...
try:
    embedder = build_embedder(settings)
    store = connect_vector_store(settings, embedder.dimension, provision=True)
except (IndexDimensionMismatch, IndexNotFound) as exc:
    ...
    return 2
```

These escape as an unhandled traceback with exit code 1:

- A missing or invalid setting, such as no `PINECONE_API_KEY` or `chunk_overlap >= chunk_size`.
- An embedding backend that fails to load.
- A Pinecone authentication or connection error.

Impact. The ingestion job is meant to run on a schedule, where exit code 1 means "some documents failed, they will be retried". A broken deployment, such as a rotated API key, therefore looks like a routine partial failure and is retried forever without raising a configuration alert. `--dry-run` is documented as touching no index, yet it crashes without a Pinecone key.

Evidence. `Finding02_ConfigErrorsExitWith2`:

- Missing API key with `--dry-run`: `'crashed with ValidationError' != 2`
- Pinecone 401 during connect: `'crashed with PineconeError' != 2`

Recommended fix. Wrap settings loading in `except ValidationError` and return 2. Widen the connect block to catch backend and Pinecone construction errors, return 2 for authentication and validation failures, and keep exit code 1 for document-level failures only. Consider letting `--dry-run` run without Pinecone credentials.

F-03 · `EMBEDDING_DIMENSIONS` can provision an index the model cannot fill

Severity: Medium · **Component:** core / embeddings · **Status:** Open, reproduced · **Effort:** S

Location: `rag/core/embeddings.py:95–96`, `:114`

Problem. The override is passed to the model only for OpenAI `text-embedding-3-*`, but it is *always* used as the embedder's declared dimension, which in turn sizes the Pinecone index:

```
if settings.embedding_dimensions and settings.openai_embedding_model.startswith("text-
embedding-3"):
    kwargs["dimensions"] = settings.embedding_dimensions
...
return Embedder(backend, ..., dimension=settings.embedding_dimensions, ...)
```

Impact. With the default `LOCAL` provider (bge-m3, 1024-dim) or `text-embedding-ada-002`, setting `EMBEDDING_DIMENSIONS` creates an index of the wrong size. Every document then fails the embedder's dimension check. Recovering requires `--allow-index-recreate`, which deletes the index. This only happens when someone sets the override, so the severity is Medium.

Evidence. `Finding03_EmbeddingDimensionsOverride`. With LOCAL and `EMBEDDING_DIMENSIONS=768`: `768 != 1024: the index would be provisioned at 768 dims while the model emits 1024`.

Recommended fix. Reject the setting in `Settings` validation unless the provider and model support it. As a fallback, ignore the override for models that cannot honour it and log a warning.

F-04 · Spreadsheet rows ignore `chunk_size`

Severity: Medium · **Component:** ingestion / parsers · **Status:** Open, reproduced · **Effort:** S

Location: `rag/ingestion/parsers.py:108–116`

Problem. The CSV/XLSX parser receives the text splitter but never uses it. Every row becomes exactly one chunk, however long:

```
chunks.append(Chunk(text=" | ".join(items), metadata=metadata))
```

Impact. A row with long free-text cells, such as a procedure column, produces an oversized chunk. Embedding models silently truncate input above their token limit (bge-m3 accepts 8,192 tokens), so the end of the row is never searchable. Above Pinecone's 40 KB per-vector metadata limit, where the chunk text is stored, the upsert is rejected and the whole workbook fails on every run. The current SOPs are Markdown, so this needs a tabular SOP with long cells to trigger, hence Medium.

Evidence. `Finding04_TableRowsNotSplit`. A one-row CSV with a long procedure cell at `chunk_size=600`: `5420 not less than or equal to 600`.

Recommended fix. Pass each row's text through `splitter.split_text()`. Repeat the row's key columns (for example the step ID) in every piece so each one stays self-describing, and carry `row_index` / `sheet` metadata on each piece.

F-05 · `--workers` bypasses validation

Severity: Low · **Component:** ingestion CLI · **Status:** Open, reproduced · **Effort:** S

Location: `rag/ingestion/cli.py:37–41`

Problem. CLI overrides are applied with `model_copy(update=...)`, which skips pydantic validation, so the `gt=0, le=32` limit on `ingest_workers` is not enforced. A value of `0` is also silently ignored because of the truthiness check:

```
if args.workers:
    overrides["ingest_workers"] = args.workers
settings = settings.model_copy(update=overrides)
```

Impact. `--workers -1` passes `max_workers=-1` to `ThreadPoolExecutor`, which raises an error, but only *after* deleted documents have already been purged. `--workers 500` is accepted even though the configured maximum is 32. This takes an operator typo to trigger, hence Low.

Evidence. `Finding05_WorkersNotValidated`. `-1`, `0` and `500` all produce `0 != 2` (accepted when they should be rejected).

Recommended fix. Rebuild the settings through validation, for example with `Settings.model_validate({**settings.model_dump(), **overrides})`, or give argparse a bounded `type=` for `--workers`. Check `is not None` instead of truthiness.

F-06 · Invalid metadata filter reported as a backend outage (503)

Severity: Medium · **Component:** retrieval API · **Status:** Open, reproduced · **Effort:** M

Location: `rag/retrieval/api.py:91–94`

Problem. Every exception other than `ValueError` becomes `503 retrieval backend unavailable` with `Retry-After: 2`, including Pinecone 400 errors caused by the caller's own `filter`:

```
except Exception as exc:
    logger.exception("Retrieval failed")
    raise HTTPException(status_code=503, detail="retrieval backend unavailable", headers=
{"Retry-After": "2"}) from exc
```

Impact. Clients and load balancers treat a caller bug as an outage: they retry it, it shows up as unavailability on dashboards, and it may trip circuit breakers. Callers get no message explaining what is wrong with their request. Today's agent tool never sends a `filter`, so this affects direct API callers, hence Medium.

Evidence. `Finding06_BadFilterReportedAsOutage`. A Pinecone 400 error ("Invalid filter operator") is returned as `503 not found in (400, 422)`.

Recommended fix. Reuse `is_transient()` from `rag/core/resilience.py`. Map transient errors to 503 and Pinecone 4xx errors to 400/422 with the upstream message. Optionally validate filter operators against an allow-list in `RetrieveRequest`.

F-07 · Agent tool misreports caller errors and crashes on malformed responses

Severity: High · **Component:** retrieval client / agent tool · **Status:** Open, reproduced · **Effort:** S

Location: `rag/retrieval/client.py:47–52`, `:71`

Problem. The client turns every HTTP error, including a 422 rejection of its *own* request, into `RetrievalUnavailable`. It also does not catch `TypeError` or missing keys when parsing the response:

```
response.raise_for_status()
return response.json()["results"]
except (httpx.HTTPError, KeyError, ValueError) as exc:
    raise RetrievalUnavailable(str(exc)) from exc
```

Impact. This is the code path the LangGraph agent depends on.

- If the LLM sends a query over 2,000 characters, or one that is only whitespace, the tool tells the operator that the SOP knowledge base is down. That is wrong, and it may lead someone to escalate an outage that doesn't exist.
- A proxy or misrouted URL that returns a JSON list or an error page makes `response.json()["results"]` raise `TypeError`. The error escapes the tool and crashes the agent step instead of degrading gracefully as the design document (TDD §2.2) requires.

Evidence. `Finding07_ClientMisreportsErrors`:

- A 3,000-character query returns `SOP_KNOWLEDGE_BASE_UNAVAILABLE...` (the 422 is reported as an outage).
- An HTTP 200 response with body `[]`: `tool raised TypeError instead of degrading gracefully`.

Recommended fix. Treat 4xx responses separately and return a distinct sentinel, for example `SOP_QUERY_REJECTED`, telling the LLM to shorten or rephrase the query. Truncate the query client-side to the service's 2,000-character limit. Validate the response shape, or catch `TypeError` along with `KeyError`, so every failure leads to a fallback message, never an exception.

F-09 · `/readyz` reports ready when the namespace is empty

Severity: Medium · **Component:** retrieval API · **Status:** Open, reproduced · **Effort:** S

Location: `rag/retrieval/api.py:70–79`

Problem. The readiness check only proves that Pinecone is reachable. It returns 200 even when the configured namespace holds zero vectors.

Impact. A wrong `PINECONE_NAMESPACE`, or a new environment where ingestion has not run yet, passes readiness and receives traffic. Every query returns no hits, so the agent answers `NO_MATCHING_SOP` to every procedure question while the deployment looks healthy.

Evidence. `Finding09_ReadyWithEmptyIndex`. An empty namespace returns `200 != 503`.

Recommended fix. Return 503 when `vector_count == 0`. If an empty index is valid in some environment, gate the check behind a setting such as `RETRIEVAL_REQUIRE_VECTORS=true`.

F-10 · Plan building is O(n²) in the number of files

Severity: Low · **Component:** ingestion · **Status:** Open, reproduced · **Effort:** S

Location: `rag/ingestion/pipeline.py:88`

Problem. `set(to_process)` is rebuilt for every file in the comprehension:

```
unchanged=[n for n in files if n not in set(to_process)],
```

Impact. This is negligible at today's scale (one SOP file) and only noticeable with thousands of documents. It is reported for completeness.

Evidence. `Finding10_QuadraticPlan`. Building the plan for 200 files calls `set()` 202 times: `202 not less than or equal to 5`.

Recommended fix. Compute `to_process_set = set(to_process)` once, or build `to_process` and `unchanged` in a single pass.

6. Withdrawn Finding

F-08 · Index recreate races Pinecone deletion (withdrawn)

Original claim. With `ALLOW_INDEX_RECREATE`, `connect_vector_store` (`rag/core/vector_store.py:115–122`) calls `pc.create_index(name)` right after `pc.delete_index(name)`. If Pinecone deletes indexes asynchronously, the create would fail with a 409 name conflict *after* the old index and all its vectors were already gone.

Why it was withdrawn. In the pinned SDK version (`pinecone==7.3.0`), `delete_index` waits by default until the deletion has finished (`pinecone/db_control/resources/sync/index.py:175–184`):

```
def delete(self, *, name: str, timeout: Optional[int] = None) -> None:
    self._index_api.delete_index(name)
    ...
    if timeout is None:
        while self.has(name=name):
            time.sleep(5)
```

`rag/` calls `delete_index(name)` without a `timeout`, so it waits until the index is gone before `create_index` runs. The race cannot happen.

Remaining risk and guard. This guarantee depends on the SDK's default behaviour. `Finding08_RecreateRace_Withdrawn` is a **passing** test that pins the assumption: it fails if anyone adds a `timeout` argument to the `delete_index` call. Re-check this behaviour whenever the `pinecone` version is upgraded.

7. Strengths Observed

These are deliberate design choices in `rag/` that the review found to be sound. They are worth keeping as the code evolves.

- **Read/write split that enforces the security model.** Only ingestion may create indexes (`provision=True`). The retrieval service connects read-only and fails fast if the index is missing. The agent-side client needs only `RETRIEVAL_SERVICE_URL`: it holds no Pinecone credentials and doesn't import torch. This matches the TDD principle of isolating the agent from raw data.
- **Safe document updates.** New vectors are upserted *before* stale ones are pruned, so a failed update leaves the previous version searchable. This is covered by `test_failed_update_keeps_previous_vectors_and_is_retried`. F-01 is the one path that bypasses this guarantee.
- **Deterministic, ASCII-safe vector IDs** (`<doc_key>#<00000>`). They allow deletion by prefix listing, which works on Pinecone serverless indexes where delete-by-metadata-filter does not.
- **Crash-safe manifest.** The manifest is written atomically (temp file, `fsync`, `os.replace`) after every document, and it is scoped per index and namespace. Its pipeline fingerprint forces a re-embed when the model or chunking settings change, even after an interrupted run.

- **Disciplined retries.** Only transient errors (timeouts, 429, 5xx) are retried, with jittered exponential backoff. The query path uses shorter limits than ingestion.
- **Safety latch on destructive operations.** A dimension mismatch never drops an index unless `ALLOW_INDEX_RECREATE` is set explicitly.
- **Graceful degradation by design.** The agent tool returns explicit sentinel messages (`SOP_KNOWLEDGE_BASE_UNAVAILABLE`, `NO_MATCHING_SOP`) that tell the LLM not to answer from memory. F-07 covers the edge cases this misses.
- **Fast, offline tests.** Heavy imports are deferred, so `--help` and `--dry-run` start instantly. The existing 17 `rag/` tests run in about 2 seconds with no network access or model downloads. That made it cheap to reproduce every finding in this review.

8. Go/No-Go Criteria and Remediation Plan

Go criteria for deploying the ingestion job and retrieval service:

1. F-01 and F-02 are fixed.
2. Their reproduction tests pass and have been moved from `tests/review_findings_test.py` into `tests/test_rag_services.py` as regression tests.
3. The full suite (`python -m unittest discover -s tests`) is green.
4. `rag/` is committed, so the deployed code can be traced to a commit.

Remediation plan

Phase	Gate	Findings	Effort
0 · Deploy gate	Before any deployment	F-01, F-02	2 × S (under 1 day)
1 · Agent gate	Before the LangGraph agent uses `search_cold_chain_sops`	F-07; F-09 recommended here too (cheap, and it prevents silent empty deployments)	2 × S (about 1 day)
2 · Hardening	Next iteration	F-03, F-04, F-06	2 × S + 1 × M (about 2 days)
3 · Hygiene	When convenient	F-05, F-10	2 × S (under 1 day)

The total estimated effort is about **4-5 engineer-days**, of which less than one day blocks deployment.

Workflow for each fix. Fix the code, check that the matching `FindingNN_*` test now passes, move the test into the main suite, and update this document's status column from *Open* to *Fixed* with the commit hash. Once every finding is fixed, `tests/review_findings_test.py` should contain only the F-08 guard test, which then also moves to the main suite.

Appendix A. Reproducing the Findings

Environment. macOS 26.5, Python 3.12.13, `pinecone` 7.3.0, `fastapi` 0.162.7, `httpx` 0.28.1, `pydantic` 2.14.3, `langchain-core` 1.6.10. No network access or credentials are needed.

Commands (from the repository root, with the project venv active):

```
# Reproduction tests for this review (expected while findings are open: 13 failures, 1 pass)
python -m unittest discover -s tests -p "review_findings_test.py" -v

# A single finding
python -m unittest discover -s tests -p "review_findings_test.py" -k Finding01

# Main suite (stays green: the review file is excluded by the default test*.py pattern)
python -m unittest discover -s tests
```

Observed output (13 Nov 2026, against code hash `20a79f80b25e`). F-02 has two tests, F-05 runs three sub-tests and F-07 has two tests, which is why 12 tests produce 13 failures.

```
FAIL  Finding01  'sop.md' not found in {} : a document with no text must be reported as
failed
FAIL  Finding02  'crashed with ValidationError' != 2           (missing PINECONE_API_KEY)
FAIL  Finding02  'crashed with PineconeError' != 2             (Pinecone 401 on connect)
FAIL  Finding03  768 != 1024 : the index would be provisioned at 768 dims while the model
emits 1024
FAIL  Finding04  5420 not less than or equal to 600
FAIL  Finding05  0 != 2                                        (--workers -1)
FAIL  Finding05  0 != 2                                        (--workers 0)
FAIL  Finding05  0 != 2                                        (--workers 500)
FAIL  Finding06  503 not found in (400, 422) : a caller error must not look like an outage
FAIL  Finding07  'SOP_KNOWLEDGE_BASE_UNAVAILABLE: ...' == 'SOP_KNOWLEDGE_BASE_UNAVAILABLE:
...'
FAIL  Finding07  tool raised TypeError instead of degrading gracefully
ok    Finding08  (withdrawn: guard test passes)
FAIL  Finding09  200 != 503 : an empty namespace answers every query with no hits
FAIL  Finding10  202 not less than or equal to 5 : set() built 202 times for 200 files:
O(n^2)

Ran 12 tests in 0.454s
FAILED (failures=13)
```